

STS2 Design Diagrams

A SQL Tools Service that explains itself while it runs

This companion to `what_was_built.md` and `sts_refactor.md` renders the STS2 architecture visually: component layering, the event-capture fan-out, the message lifecycle through the pump, the journal-authoritative / wire-enriched pattern, the live tail, runtime config change, and the Core state machines. Colour legend used throughout: runtime component, pure Core, write-ahead journal, observer sink, async edge, external.

1. Process architecture: one channel, two services

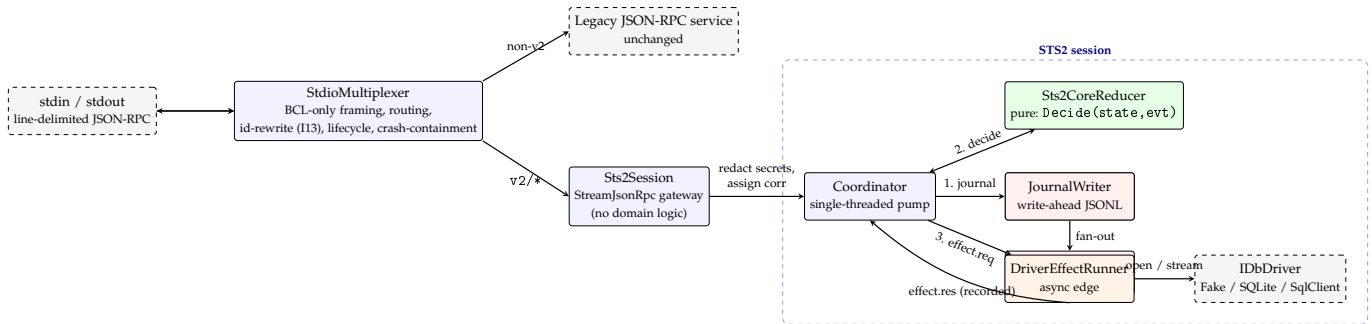


Figure 1: STS2 runs inside the existing process, beside the legacy service, sharing one stdio channel. The multiplexer routes `v2/*` to STS2 and everything else to legacy untouched. Inside the session the gateway holds no domain logic; the Coordinator pump is the only mover, and the Core reducer is the only decider.

2. The event-capture seam: write-ahead journal + best-effort fan-out

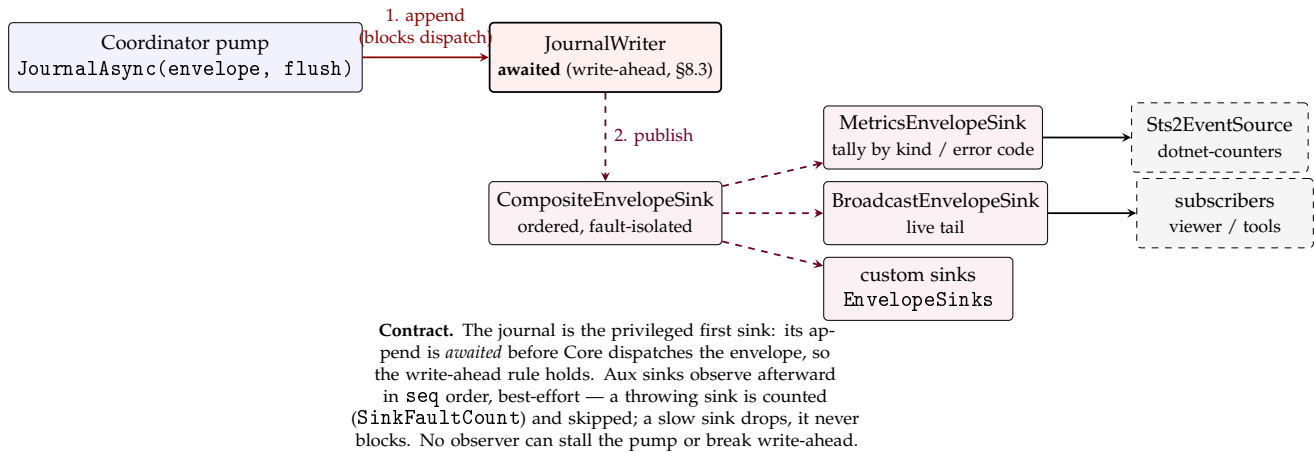


Figure 2: Observability is a fan-out seam (*IEnvelopeSink*), not a hard-wired consumer. Solid red is the synchronous write-ahead path; dashed purple is the best-effort, fault-isolated publish to auxiliary observers.

3. Message lifecycle through the pump

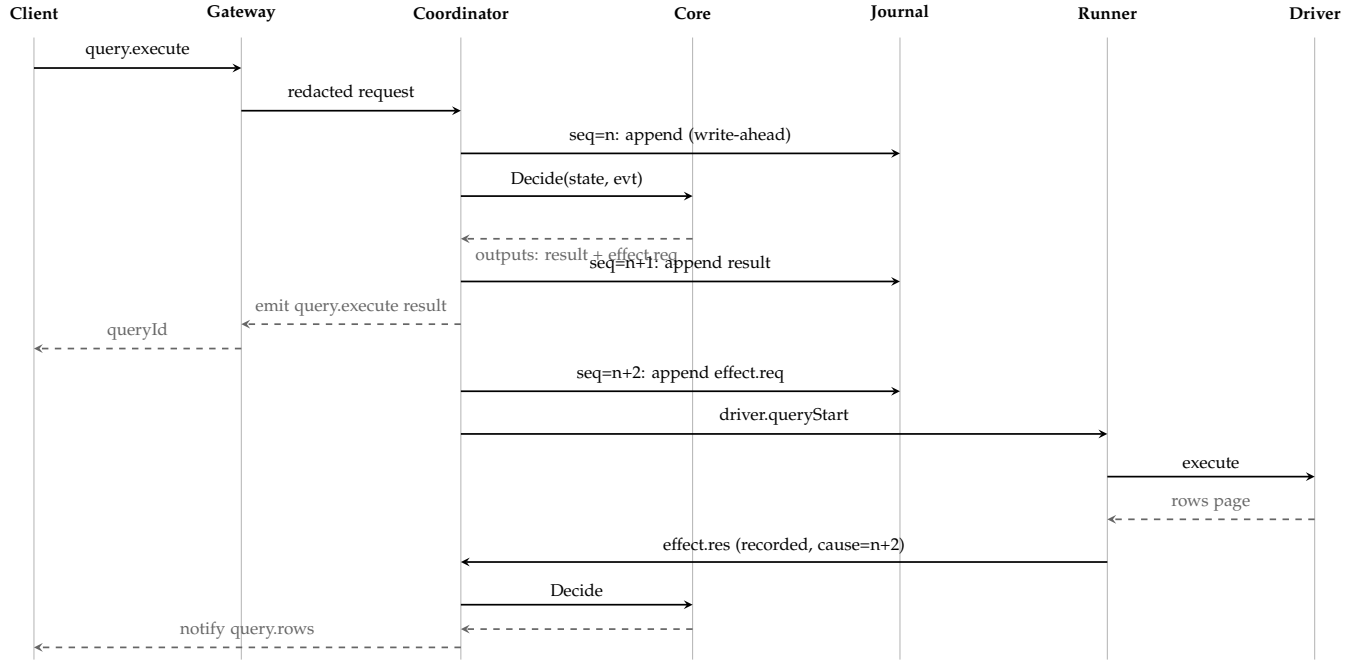
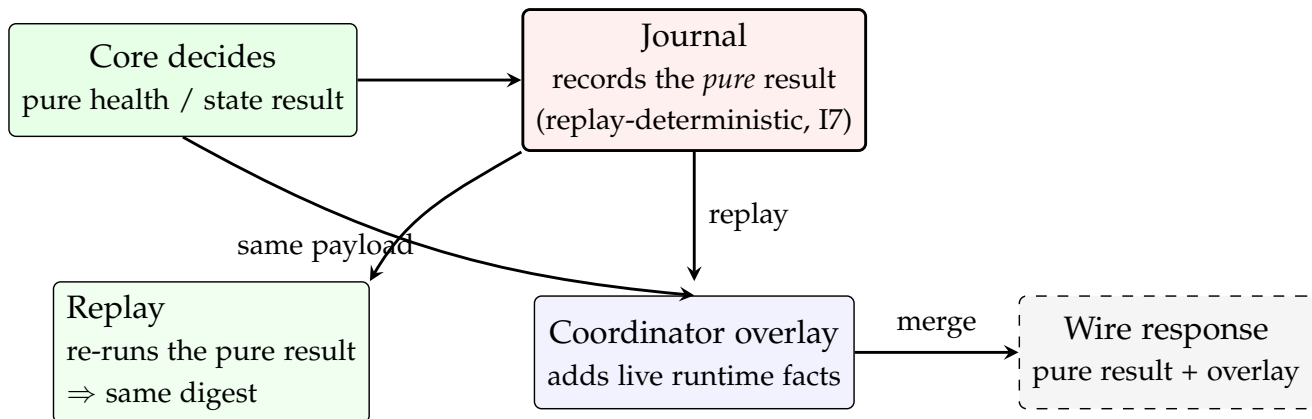


Figure 3: One execute, end to end. Every external observation (the driver's rows page) re-enters the pump as a recorded `effect.res` envelope carrying a `cause` pointer back to the effect request that produced it. Because nothing reaches Core except journaled events, the journal alone is enough to replay the run.

4. Journal-authoritative, wire-enriched: the overlay pattern



health overlay: queueDepth, openLeases / opensInFlight / activeQueryPumps, fatal+reason, configVersion, dropped-Diagnostics, recentErrors histogram, envelopesObserved.
state overlay: a runtime section (handle summaries, configVersion).
capture elision (same shape): the journal records digest *wrappers*; the wire gets the original cells/SQL substituted back.

Figure 4: Live introspection needs facts only the Runtime knows (queue depth, leases) — but those are non-deterministic, so journaling them would break replay. The fix, used by health, state, and capture elision alike: the journal records the deterministic truth; the coordinator overlays the live facts onto the wire response only. Replay sees the pure form and stays digest-identical.

5. Live tail: bounded, drop-oldest, counted

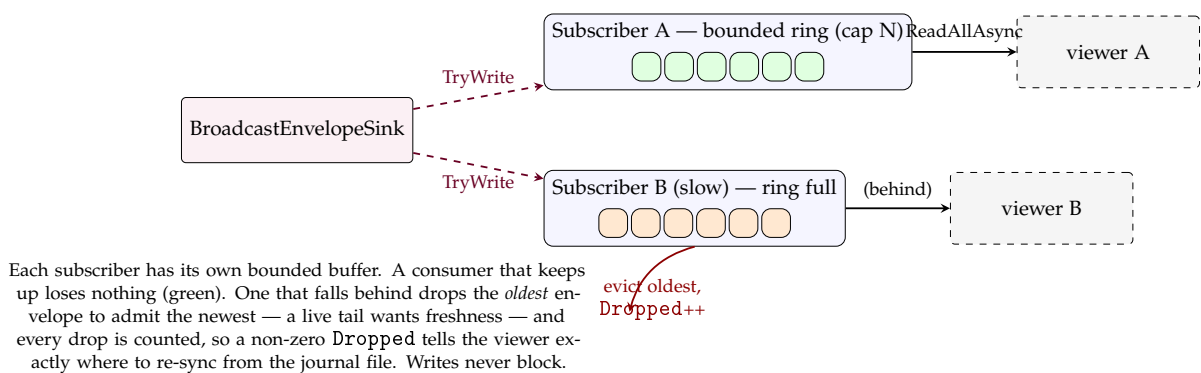
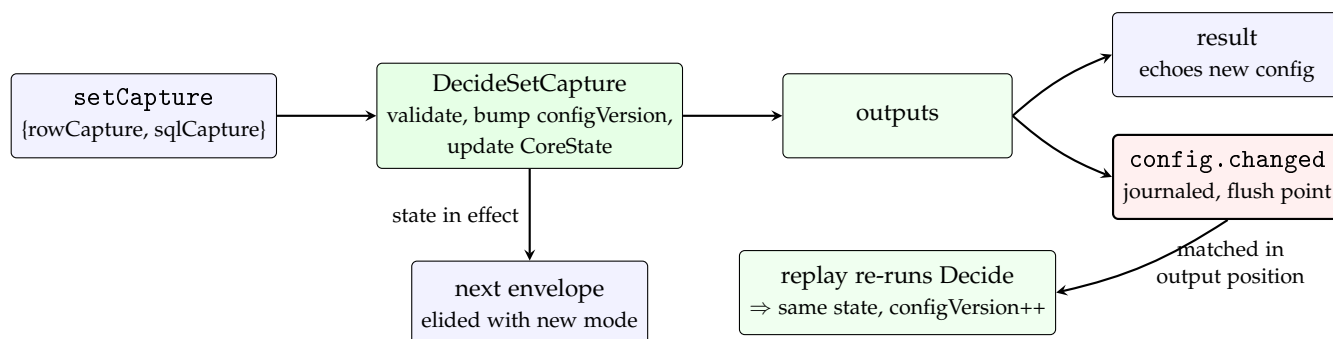


Figure 5: The diagnostic viewer's feed. Freshness over completeness, with an exact drop count so gaps are detectable and recoverable.

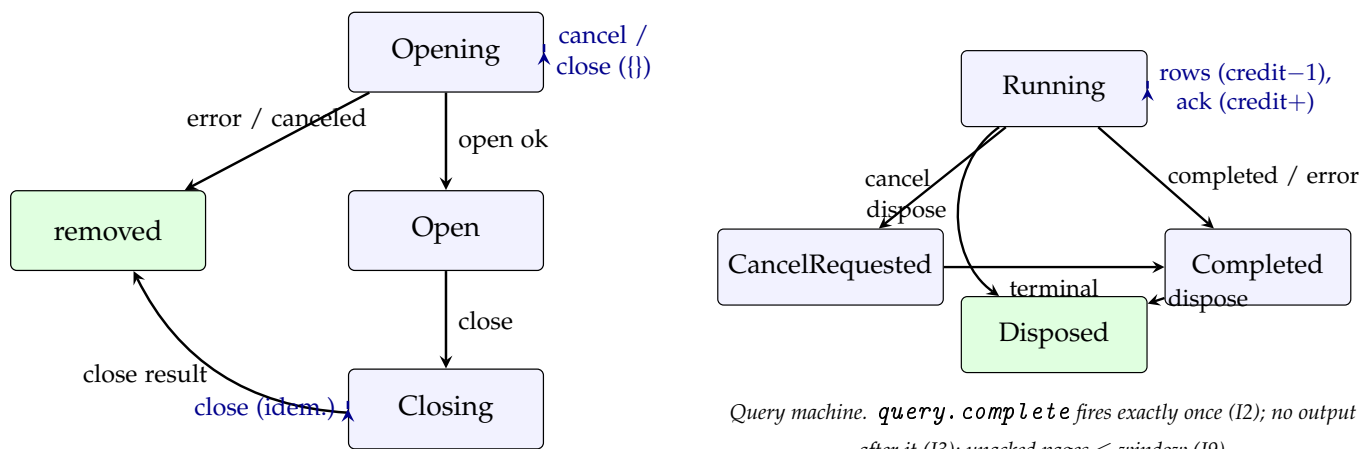
6. Runtime config change: `setCapture` → `config.changed` → **replay-visible**



Capture mode and `configVersion` live in *Core state*, seeded by the journaled `session.start` so replay starts identically, and changed here. The change takes effect on the next envelope (elision reads post-decision state); the journaled `config.changed` is matched during replay, so the new `configVersion` is visible in replayed state (invariant I15).

Figure 6: Runtime reconfiguration is itself a journaled, replayable event — not hidden mutable state. The three formerly hard-coded `configVersion=1` literals are gone; the version is sourced from *Core state* and stamped on every subsequent envelope.

7. The Core state machines



Connection machine. One active query at a time; close cancels it first.

Query machine. `query.complete` fires exactly once (I2); no output after it (I3); unacked pages ≤ window (I9).

Figure 7: The concurrency-sensitive logic lives entirely in the pure reducer, so both machines are testable as a function and reproducible by replay. Idempotent self-loops (blue) return `{}` for unknown or terminal ids.